

Relatório Técnico: Sistemas Embarcados

Identificação do Aluno	Número USP	Data de Entrega
Giovanni Antonio Silva de Oliveira	14575064	03/06/2026

1. Identificação do Experimento

- **Título:** Atividade 4 - Registradores ADC
- **Disciplina:** PSI3441 - Arquitetura de Sistemas Embarcados
- **Professor:** Gustavo Pamplona Rehder

2. Respostas, Comentários e Análises

Nesse experimento, foi feita a configuração novamente de um periférico pelo uso de escrita e configuração direta nos registradores internos do controlador, mas com um novo periférico, o Conversor Analógico-Digital (CAD ou ADC). O código foi feito tentando cumprir os critérios do enunciado do respectivo Exercício, e as orientações de configuração e funcionamento do ADC no manual de referência, e o teste usando somente um potenciômetro, com a saída de 3v3 e gnd da própria placa de prototipação.

3. Código-Fonte

```
#include <zephyr.h>

// ===== Mapa dos Registradores
// =====
//////////////////// Clock //////////////////////

#define SIM_SCGC5    (*((volatile unsigned int*)0x40048038)) // Clock
GPIO
#define SIM_SCGC6    (*((volatile unsigned int*)0x4004803C)) // Clock
ADC
```

```

//////////////////// GPIO //////////////////////

/* Multiplexador do pino*/

#define PORTB_PCR19  (*((volatile unsigned int*)0x4004A04C)) // Verde
#define PORTD_PCR1   (*((volatile unsigned int*)0x4004C004)) // Azul
#define PORTE_PCR20  (*((volatile unsigned int*)0x4004D050)) //
Analógico

//////////////////// Pino Verde (PTB19) //////////////////////

#define GPIOB_PDDR    (*((volatile unsigned int*)0x400FF054)) // Define
se é Input ou Output (direção)
#define GPIOB_PTOR    (*((volatile unsigned int*)0x400FF04C)) //
Registrador Toggle
#define GPIOB_PSOR    (*((volatile unsigned int*)0x400FF044)) // Nivel
lógico Alto
#define GPIOB_PCOR    (*((volatile unsigned int*)0x400FF048)) // Nivel
lógico Baixo

//////////////////// Pino Azul (PTD1) //////////////////////

#define GPIOD_PDDR    (*((volatile unsigned int*)0x400FF0D4)) // Define
se é Input ou Output (direção)
#define GPIOD_PTOR    (*((volatile unsigned int*)0x400FF0CC)) //
Registrador Toggle
#define GPIOD_PSOR    (*((volatile unsigned int*)0x400FF0C4)) // Nivel
lógico Alto
#define GPIOD_PCOR    (*((volatile unsigned int*)0x400FF0C8)) // Nivel
lógico Baixo

//////////////////// ADC0 //////////////////////

#define ADC0_SC1A     (*((volatile unsigned int*)0x4003B000)) // Controle
e Status
#define ADC0_CFG1     (*((volatile unsigned int*)0x4003B008)) //
Configuração
#define ADC0_RA        (*((volatile unsigned int*)0x4003B010)) //
Resultado da Conversão

//////////////////// Parametros Limite //////////////////////

```

```

#define MAX_TOL 0.9
#define MIM_TOL 0.1
#define RESOLUCAO 16

void main(void)
{
    // Ativa o clock das portas GPIO do barramento E, D e B
    SIM_SCGC5 |= (uint32_t)((0b1 << 10) | (0b1 << 12)) | (0b1 << 13);
    // Ativa o clock dos registradores do ADC
    SIM_SCGC6 |= (uint32_t)(0b1 << 27);

    //////////// configura pino led verde ////////////

    // Limpa e Configura Tipo de saída do pino (GPIO)
    PORTB_PCR19 &= ~(uint32_t)(0b111 << 8);
    PORTB_PCR19 |= (uint32_t)(0b001 << 8);
    // Direção de Saída do Pino
    GPIOB_PDDR |= (uint32_t)(0b1 << 19);
    // Nivel Inicial Alto (por ser anodo comum)
    GPIOB_PSOR |= (uint32_t)(0b1 << 19);

    //////////// configura pino led Azul ////////////

    // Limpa e Configura Tipo de saída do pino (GPIO)
    PORTD_PCR1 &= ~(uint32_t)(0b111 << 8);
    PORTD_PCR1 |= (uint32_t)(0b001 << 8);
    // Direção de Saída do Pino
    GPIOD_PDDR |= (uint32_t)(0b1 << 1);
    // Nivel Inicial Alto (por ser anodo comum)
    GPIOD_PSOR |= (uint32_t)(0b1 << 1);

    //////////// Configuração ADC ////////////

    // deixa no modo analogico
    PORTE_PCR20 &= ~(uint32_t)(0b111 << 8);
    // Resolução Máxima, Fonte de clock interna e sem dividir clock de
    entrada
    ADC0_CFG1 |= (uint32_t)((0b11 << 2) | (0b00 << 0) | (0b00 << 5));

    for(;;)
    {
        // Inicia a Conversão
        ADC0_SC1A = 0x00000000;
    }
}

```

```

        // Aguarda Flag de conversão completa
        while (!(ADC0_SC1A & (0b1 << 7)));
        // Estrai valor do registrador (de 0 a 65535, dado a resolução
de 16 bits)
        uint16_t adc_value = (uint16_t)ADC0_RA;

        if (adc_value > (uint16_t)((0b1 << RESOLUCAO) - 1)*MAX_TOL))
        {
            GPIOD_PCOR = (uint32_t)(0b1 << 1); // Acende azul
            GPIOB_PSOR = (uint32_t)(0b1 << 19); // Apaga Verde
        }
        else if (adc_value < (uint16_t)((0b1 << RESOLUCAO) -
1)*MIM_TOL))
        {
            GPIOD_PSOR = (uint32_t)(0b1 << 1); // Apaga azul
            GPIOB_PCOR = (uint32_t)(0b1 << 19); // Acende Verde
        }
        else
        {
            // Todos Apagados
            GPIOB_PSOR = (1 << 19);
            GPIOD_PSOR = (1 << 1);
        }
    }
}

```

4. Referências e Links Externos

Utilize este campo para referenciar repositórios de controle de versão, vídeos demonstrativos ou documentos complementares.

- **Link do Repositório (GitHub/GitLab):**

<https://github.com/Djovanne/PSI3441---Arquitetura-de-Sistemas-Embarcados/tree/main/Atividade%204%20-%20ADC>